Solution du DS 2

CPES « Sciences des données, arts et cultures » : Introduction à l'algorithmique

Antoine GAUQUIER antoine.gauquier@ens.fr
Pierre SENELLART pierre@senellart.com

16 janvier 2024

Multiplication de polynômes

1. (2 points)
```
class Polynome:
    def __init__(self, coeff = []):
        if coeff == [0]:
            coeff = []
        self._degre = len(coeff) - 1
        self._coeff = coeff
```
2. (1 point)
```
    def __getitem__(self, x):
        if(self._degre == -1):
            return 0
        return self._coeff[x]
```
3. (2 points)
```
    def __call__(self, x):
        value = 0
        for coeff in reversed(self._coeff):
            value *= x
            value += coeff
        return value
```
   La complexité est en $O(d)$.
4. (3 points)
```
    def __str__(self):
        if self._degre == -1:
            return "0"

        l = []
        first = True
        for i in reversed(range(0, self._degre + 1)):
            if self[i] != 0:
                if first:
                    first = False
                    if self[i]<0:
```

```
                l.append("-")
            else:
                l.append("+" if self[i]>0 else "-")
            if i==0:
                monomial = ""
            elif i==1:
                monomial = "X"
            else:
                monomial = "X^" + str(i)

            if abs(self[i])==1 and i!=0:
                l.append(monomial)
            else:
                l.append(str(abs(self[i])) + monomial)

        return " ".join(l)
```

5. (2 points)

```
    def __add__(self, other):
        coeff = [0 for i in range(0, max(self._degre, other._degre) + 1)]
        for i in range(0, len(coeff)):
            if(i<=self._degre):
                coeff[i] += self[i]
            if(i<=other._degre):
                coeff[i] += other[i]
        return Polynome(coeff)
```

   La complexité est en $O(\max(d_1, d_2))$.

6. (1 point)

```
    def __sub__(self, other):
        coeff = [0 for i in range(0, max(self._degre, other._degre) + 1)]
        for i in range(0, len(coeff)):
            if(i<=self._degre):
                coeff[i] += self[i]
            if(i<=other._degre):
                coeff[i] -= other[i]
        return Polynome(coeff)
```

   La complexité est en $O(\max(d_1, d_2))$.

7. (2 points)

```
    def __mul__(self, other):
        coeff = [0 for i in range(0, self._degre + other._degre + 1)]
        for i in range(0, self._degre+1):
            for j in range(0, other._degre+1):
                coeff[i+j] += \
                  self[i] * other[j]
        return Polynome(coeff)
```

   La complexité est en $O(d_1 \times d_2)$.

8. (2 points) Soit $d = \max(d_1, d_2)$ le degré maximal des deux polynômes. On obtient la récurrence suivante pour le temps $T(d)$ d'exécution de l'algorithme **B** en fonction de d, en faisant l'hy-

pothèse que d est une puissance de 2 :

$$\begin{cases} T(0) = O(1) \\ T(d) = 4T\left(\frac{d}{2}\right) + \Theta(d) \end{cases}$$

On utilise le théorème maître, avec $a = 4$, $b = 2$, $c = \log_2 4 = 2$ et $f(d) = \Theta(d^1)$. On est dans le premier cas du théorème maître et donc $T(d) = \Theta(d^c) = \Theta(d^2)$. C'est la même complexité que pour l'algorithme **N**.

9. (2 points) On peut calculer R_{12} avec $R_{12} = (P_1 + P_2) \times (Q_1 + Q_2) - R_1 - R_2$ donc avec une unique multiplication. Ça donne un algorithme diviser pour régner avec 3 multiplications (pour calculer chacun de R_1, R_{12}, R_2) et des additions de polynômes, donc une complexité de :

$$\begin{cases} T(0) = O(1) \\ T(d) = 3T\left(\frac{d}{2}\right) + \Theta(d) \end{cases}$$

On utilise le théorème maître, avec $a = 3$, $b = 2$, $c = \log_2 3 = 2$ et $f(d) = \Theta(d^1)$. On est dans le premier cas du théorème maître et donc $T(d) = \Theta(d^c) = \Theta(d^{\log_3 2})$. C'est une meilleure complexité que celle de l'algorithme **N**.

10. (3 points)

```
def algok(p, q):
    degre = max(p._degre, q._degre)
    m = -(-degre // 2)
    if(m==0):
        return Polynome([p[0]*q[0]])
    else:
        p1 = Polynome(p[0:m])
        p2 = Polynome(p[m:])
        q1 = Polynome(q[0:m])
        q2 = Polynome(q[m:])
        r1  = Polynome.algok(p1, q1)
        r2  = Polynome.algok(p2, q2)
        r12 = Polynome.algok(p1+p2,q1+q2) - r1 - r2
        coeff = [0 for i in range(0, p._degre + q._degre + 1)]
        for i in range(0, r1._degre+1):
            coeff[i] += r1[i]
        for i in range(0, r12._degre+1):
            coeff[i+m] += r12[i]
        for i in range(0, r2._degre+1):
            coeff[i+2*m] += r2[i]
        return Polynome(coeff)
```